MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 9 — Les Tuples

L'immuabilité comme force — quand figer les données a du sens

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

7 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 9 — Les Tuples **2**
0.1 Créer un tuple 2
0.2 Accéder aux éléments 3
0.3 L'immuabilité — Force et limites 3
0.4 Le déballage — Unpacking 4
0.5 Tuples et fonctions 5
0.6 Tuple vs Liste — Quand choisir ? 6
0.7 Les namedtuples — Tuples avec des noms 6
0.8 Les tuples comme clés de dictionnaire 7
0.9 Ce qui m'a le plus surpris dans ce chapitre 8
0.10 Résumé du Chapitre 9 8

Chapitre 9 — Les Tuples

À retenir

Un tuple est une **séquence ordonnée et immuable**. Comme une liste, mais qu'on ne peut pas modifier après création. Cette contrainte n'est pas une faiblesse — c'est une **garantie**. Quand une donnée ne doit jamais changer, le tuple est le choix naturel. Plus rapide, plus sûr, plus expressif que la liste dans ces cas précis.

Créer un tuple

Un tuple est délimité par des **parenthèses** `()` — ou même sans parenthèses dans certains cas.

Créer des tuples

```python
# Avec parentheses
coordonnees = (48.8566, 2.3522)    # Paris      lat, lon
couleur_rgb = (255, 128, 0)        # Orange en RGB
point = (3, 7)

# Sans parentheses     Python comprend quand meme
dimensions = 1920, 1080
print(dimensions)         # (1920, 1080)
print(type(dimensions))   # <class 'tuple'>

# Tuple d'un seul element      la virgule est OBLIGATOIRE
seul = (42,)
print(type(seul))     # <class 'tuple'>
pas_tuple = (42)
print(type(pas_tuple))  # <class 'int'>      juste des
    parentheses !

# Tuple vide
vide = ()
print(type(vide))     # <class 'tuple'>

# Depuis une liste
liste = [1, 2, 3]
t = tuple(liste)
print(t)    # (1, 2, 3)
```

Erreur fréquente

Le tuple à un seul élément nécessite une **virgule** : `(42,)` et non `(42)`. Sans la virgule, Python interprète les parenthèses comme de simples parenthèses mathématiques — pas comme un tuple. C'est l'une des erreurs les plus subtiles et les plus fréquentes avec les tuples.

Accéder aux éléments

L'accès aux éléments d'un tuple est identique à celui des listes — index et slicing fonctionnent de la même façon.

Acces et slicing

```python
point = (10, 20, 30, 40, 50)

# Index positifs et negatifs
print(point[0])      # 10
print(point[-1])     # 50
print(point[2])      # 30

# Slicing
print(point[1:4])    # (20, 30, 40)
print(point[:3])     # (10, 20, 30)
print(point[::-1])   # (50, 40, 30, 20, 10)

# Longueur
print(len(point))    # 5

# Recherche
print(30 in point)        # True
print(point.index(30))    # 2
print(point.count(20))    # 1

#     Modification impossible     TypeError
# point[0] = 99
```

L'immuabilité — Force et limites

Immuabilité du tuple

```python
coordonnees = (48.8566, 2.3522)

#     Impossible de modifier
# coordonnees[0] = 0.0        TypeError

#     Impossible d'ajouter
# coordonnees.append(100)        AttributeError
```

```python

#     On peut creer un NOUVEAU tuple
nouvelles = coordonnees + (100,)
print(nouvelles)   # (48.8566, 2.3522, 100)

#     Concatenation et repetition
t1 = (1, 2, 3)
t2 = (4, 5, 6)
print(t1 + t2)    # (1, 2, 3, 4, 5, 6)
print(t1 * 3)     # (1, 2, 3, 1, 2, 3, 1, 2, 3)

# Tuple contenant une liste     subtilite !
bizarre = (1, 2, [3, 4])
bizarre[2].append(5)     # La LISTE interieure est mutable
print(bizarre)   # (1, 2, [3, 4, 5])      le tuple "change" !
```

Ce qui m'a résisté

La dernière subtilité m'a vraiment intrigué : un tuple peut *sembler* changer si l'un de ses éléments est une liste mutable. Le tuple lui-même ne change pas — ses références restent les mêmes. Mais l'objet *pointé* par l'une de ces références peut être modifié. C'est la différence entre **immuabilité de la structure** et **immuabilité du contenu** — une nuance profonde sur la gestion de la mémoire en Python.

Le déballage — Unpacking

L'**unpacking** est l'une des fonctionnalités les plus élégantes des tuples — et de Python en général.

Unpacking — déballage de tuples

```python
# Deballage simple
point = (10, 20)
x, y = point
print(x)   # 10
print(y)   # 20

# Deballage d'une coordonnee GPS
paris = (48.8566, 2.3522, "Paris")
latitude, longitude, ville = paris
print(f"{ville} : {latitude} N , {longitude} E ")

# Ignorer des valeurs avec _
donnees = (2024, 3, 15, "vendredi")
annee, mois, jour, _ = donnees
print(f"{jour}/{mois}/{annee}")   # 15/3/2024

# Deballage etendu avec *
```

```python
premier, *milieu, dernier = (1, 2, 3, 4, 5)
print(premier)    # 1
print(milieu)     # [2, 3, 4]
print(dernier)    # 5

# Swap elegant (deja vu au chapitre 2 !)
a, b = 10, 20
a, b = b, a
print(a, b)    # 20 10
```

À retenir

L'unpacking avec `*` (étoile) est puissant : il capture **tous les éléments restants** dans une liste. `premier, *reste = (1, 2, 3, 4, 5)` donne `premier = 1` et `reste = [2, 3, 4, 5]`. C'est une syntaxe que l'on retrouve aussi dans les fonctions avec `*args`.

Tuples et fonctions

Les tuples sont très utilisés pour **retourner plusieurs valeurs** depuis une fonction.

Retourner plusieurs valeurs

```python
# Une fonction peut retourner plusieurs valeurs
# Python les emballe automatiquement dans un tuple
def min_max(liste):
    return min(liste), max(liste)    # retourne un tuple

notes = [15, 8, 18, 5, 12, 17]
resultat = min_max(notes)
print(resultat)           # (5, 18)
print(type(resultat))     # <class 'tuple'>

# Deballage direct a l'appel
minimum, maximum = min_max(notes)
print(f"Min : {minimum}, Max : {maximum}")

# Exemple plus complet
def statistiques(liste):
    n = len(liste)
    total = sum(liste)
    moyenne = total / n
    return n, total, moyenne, min(liste), max(liste)

n, total, moy, mini, maxi = statistiques(notes)
print(f"N={n}, Total={total}, Moy={moy:.1f}")
print(f"Min={mini}, Max={maxi}")
```

Tuple vs Liste — Quand choisir ?

TUPLE ()	LISTE []
Immuable	Mutable
Plus rapide en lecture	Modifiable à volonté
Moins de mémoire	Méthodes riches
Utilisable comme clé de dict	Non utilisable comme clé
Utiliser pour :	**Utiliser pour :**
Coordonnées GPS	Panier d'achats
Couleurs RGB	Liste de tâches
Retours de fonctions	Résultats d'une requête
Données constantes	Collections dynamiques
Records de données	Données à trier/filtrer

Les namedtuples — Tuples avec des noms

`namedtuple` permet de créer des tuples dont les champs ont des **noms explicites** — rendant le code beaucoup plus lisible.

namedtuple — tuples nommés

```python
from collections import namedtuple

# Definir un type namedtuple
Point = namedtuple("Point", ["x", "y"])
Personne = namedtuple("Personne", ["prenom", "nom", "age"])
Couleur = namedtuple("Couleur", ["rouge", "vert", "bleu"])

# Creer des instances
p = Point(10, 20)
alice = Personne("Alice", "Dupont", 25)
orange = Couleur(255, 128, 0)

# Acceder par NOM      beaucoup plus lisible !
print(p.x, p.y)                # 10 20
print(alice.prenom)             # Alice
print(alice.age)                # 25
print(f"RGB({orange.rouge}, {orange.vert}, {orange.bleu})")

# Acces par index aussi possible
print(alice[0])    # Alice
print(alice[2])    # 25

# Deballage fonctionne toujours
prenom, nom, age = alice
```

```python
print(f"{prenom} {nom}, {age} ans")

# Conversion en dictionnaire
print(alice._asdict())
# OrderedDict([('prenom', 'Alice'), ('nom', 'Dupont'), ('age',
    25)])
```

À retenir

Le `namedtuple` est un compromis élégant entre le tuple et la classe. Il est **immuable comme un tuple** mais **lisible comme un objet**. Accéder à `alice.prenom` est infiniment plus clair que `alice[0]`. C'est la solution pythonique pour représenter des données structurées simples sans créer une classe complète.

Les tuples comme clés de dictionnaire

Contrairement aux listes, les tuples peuvent être utilisés comme **clés de dictionnaire** car ils sont immuables et donc *hashables*.

Tuples comme clés de dict

```python
# Les listes ne peuvent PAS etre cles de dict
#     d = {[1,2]: "valeur"}      TypeError: unhashable type

# Les tuples LE PEUVENT
capitales = {
    (48.8566, 2.3522): "Paris",
    (51.5074, -0.1278): "Londres",
    (40.7128, -74.0060): "New York"
}

# Chercher par coordonnees
coord = (48.8566, 2.3522)
print(capitales[coord])   # Paris

# Grille de jeu (ligne, colonne)     valeur
plateau = {
    (0, 0): "X", (0, 1): " ", (0, 2): "O",
    (1, 0): " ", (1, 1): "X", (1, 2): " ",
    (2, 0): "O", (2, 1): " ", (2, 2): "X"
}
print(plateau[(1, 1)])   # X
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. La virgule fait le tuple — pas les parenthèses. `(42)` n'est pas un tuple. `42,` l'est. `(42,)` aussi. Cette réalité m'a semblé contre-intuitive pendant un moment. Les parenthèses ne créent pas un tuple — elles l'encadrent. C'est la **virgule** qui signale à Python qu'il s'agit d'une séquence. Un détail minuscule, une conséquence potentiellement massive.

2. Un tuple peut "changer" si son contenu est mutable. `(1, 2, [3, 4])` — le tuple est immuable, mais la liste à l'intérieur ne l'est pas. On peut modifier la liste, et le tuple semble avoir changé. Ce n'est pas le tuple qui change — c'est l'objet vers lequel il pointe. Cette distinction entre la référence et l'objet pointé m'a fait replonger dans la façon dont Python gère la mémoire.

3. Le `namedtuple` est une révélation. Avant ce chapitre, j'aurais créé une classe entière pour représenter un point ou une personne. Le `namedtuple` offre la lisibilité d'un objet, la performance d'un tuple, et la simplicité d'une ligne de code. C'est le genre de fonctionnalité qui montre à quel point Python a été pensé pour rendre la vie du développeur **agréable**.

Résumé du Chapitre 9

Ce que j'ai appris dans ce chapitre :

- Un tuple est une séquence **ordonnée et immuable** — délimitée par `()`.
- La **virgule** fait le tuple — pas les parenthèses : `(42,)` `(42)`.
- **Unpacking** : `x, y = (10, 20)` — déballage élégant des valeurs.
- **Unpacking étendu** : `premier, *reste = tuple`.
- Les fonctions retournent plusieurs valeurs via des tuples implicites.
- Les tuples sont **hashables** — utilisables comme clés de dictionnaire.
- `namedtuple` — tuple avec des champs nommés, lisible comme un objet.
- Choisir le tuple quand la donnée **ne doit pas changer** — liste sinon.

"L'immuabilité n'est pas une contrainte. C'est une promesse — celle que cette donnée ne changera jamais."

KETOTSA AMÉVI CLAUDE — Mon apprentissage Python | Chapitre 9 sur 34